

Test Cases Document — Horizon AI

Comprehensive test specification for unit and integration testing

Table of Contents

1. Section 1: Methodology

- 1.1 Testing Principles
- 1.2 Test Classification
- 1.3 Mock Strategy
- 1.4 Pass Criteria

2. Section 2: Backend Unit Tests

- 2.1 OrchestrationConfig
- 2.2 Types & Schemas
- 2.3 ResponseParser
- 2.4 ASTMatcher
- 2.5 Validator
- 2.6 AgentService
- 2.7 ClientConnection
- 2.8 DatabaseManager
- 2.9 MessageRouter
- 2.10 Orchestrator
- 2.11 Phase 2 — Strategy
- 2.12 Phase 3 — Execution
- 2.13 Phase 4 — Validation
- 2.14 Phase 5 — Adjudication
- 2.15 Phase 6 — Finalization
- 2.16 Formatters
- 2.17 Code Utils

3. Section 3: Backend Integration Tests

4. Section 4: Frontend Unit Tests

- 4.1 lib/utils — cn()
- 4.2 lib/constants
- 4.3 lib/parseStatusInfo
- 4.4 lib/formatStatusContent
- 4.5 lib/javaFormatter
- 4.6 lib/indentation
- 4.7 lib/buildMetrics
- 4.8 lib/schemas/websocket (Zod)
- 4.9 store/useChatStore
- 4.10 hooks/useOrchestrationSocket
- 4.11 components/ErrorBoundary

5. Section 5: Frontend Integration Tests

- 5.1 Terminal
- 5.2 ChatWorkspace

6. Section 6: Test Execution Matrix

Project Horizon AI — Multi-Agent Java Refactoring Pipeline

Prepared by Horizon QA Team

Date June 24, 2026

Version 1.0

Purpose Verify correctness, reliability, and edge-case resilience of all backend and frontend modules through isolated unit tests and integrated API, WebSocket, and component-level testing.

Section 1: Methodology

1.1 Testing Principles

This test suite follows the **AAA (Arrange-Act-Assert)** pattern:

Step	Description
Arrange	Set up preconditions, create mocks, prepare input data
Act	Execute the function or method under test
Assert	Verify the output, side effects, or state changes

1.2 Test Classification

Type	Definition
Positive	Valid input produces correct output
Negative	Invalid input produces expected error/failure
Edge	Boundary conditions, empty/null inputs, edge cases

1.3 Mock Strategy

Module	Mock Level	Dependencies Mocked
OrchestrationConfig	None (pure)	—
Types & Schemas	None (pure)	—
ResponseParser	None (pure)	—
ASTMatcher	None (pure)	—
Validator	None (pure)	External libraries (javalang, lizard) used real
AgentService	Function-level	llama_cpp.Llama, file I/O
ClientConnection	Class-level	WebSocket send_json, database
DatabaseManager	Integration	Real in-memory SQLite (no mock)
MessageRouter	Class-level	AgentService, ClientConnection
Orchestrator	Class-level	AgentService, ClientConnection, DatabaseManager
Phase 2 — Strategy	Class-level	AgentService.generate,

		ASTMatcher
Phase 3 — Execution	Class-level	AgentService.generate, Validator
Phase 4 — Validation	Class-level	Validator, OrchestrationState
Phase 5 — Adjudication	Class-level	AgentService.generate
Phase 6 — Finalization	Class-level	AgentService.generate, DatabaseManager
Formatters	None (pure)	—
Code Utils	None (pure)	—
Frontend store	Integration	Real Zustand state, mock fetch
Frontend hooks	Class-level	WebSocket constructor, store

1.4 Pass Criteria

- All **Positive** tests pass: function returns correct value
- All **Negative** tests pass: function raises or returns error as expected
- All **Edge** tests pass: function handles boundaries gracefully
- **No test** may depend on another test (no shared mutable state between cases)

Section 2: Backend Unit Tests

2.1 OrchestrationConfig

Module: app/modules/orchestrator/config.py

Purpose: Validate YAML/JSON config loading, deserialization, and immutability of model configuration entries.

Mock strategy: Pure functions. No mocking. Test data loaded from dict.

ID	Type	Scenario	Input	Expected
TC-CFG-001	Positive	from_dict with valid config creates OrchestrationConfig	{"planner": {...}, "generator": {...}, "judge": {...}, "single": {...}, "settings": {...}}	Returns populated OrchestrationConfig
TC-CFG-002	Negative	from_dict missing required key raises validation error	{"planner": {...}} (missing generator, judge, single)	Raises TypeError OR KeyError
TC-CFG-003	Positive	from_yaml loads valid YAML file	Path to valid YAML	Returns OrchestrationConfig
TC-CFG-004	Negative	from_yaml invalid YAML raises	Path to malformed YAML	Raises exception
TC-CFG-005	Edge	ModelEntry frozen dataclass cannot be mutated	Create ModelEntry, attempt to modify temperature	Raises FrozenInstanceError

2.2 Types & Schemas

Module: app/utils/types.py, app/utils/schemas.py

Purpose: Validate Pydantic model constraints, enum validation, and request/schema structural rules.

Mock strategy: Pure Pydantic validation. No mocking.

ID	Typ	Scenario	Input	Expected
TC-TS-001	Positive	RefactorRequest(code, user_instruction) valid	{ "type": "multi", "code": "class A {}", "user_instruction": "refactor"} }	Passes validation
TC-TS-002	Negative	RefactorRequest empty code rejected	{ "type": "multi", "code": "", "user_instruction": "refactor"} }	Validation error
TC-TS-003	Edge	RefactorRequest minimum instruction length	{ "type": "multi", "code": "class A {}", "user_instruction": "ab"} }	Validation error (< 3 chars)
TC-TS-004	Positive	IntentPacket with valid enums	{ "refactor_category": "CONTROL_FLOW", "specific_intent": "FLATTEN_CONDITIONAL", "scope_anchor": {"class": "A", "unit_type": "METHOD_UNIT"}} }	Passes validation
TC-TS-005	Negative	IntentPacket invalid enum value	{ "refactor_category": "INVALID", "specific_intent": "FLATTEN_CONDITIONAL"} }	Validation error
TC-TS-006	Positive	ASTMutation with max 5 mutations	Array of 5 ASTMutation items	Passes validation
TC-TS-007	Negative	ASTMutation more than 5 mutations rejected	Array of 6 ASTMutation items	Validation error (max_length=5)
TC-TS-008	Edge	ScopeAnchor with optional member field null	{ "class": "A", "member": null, "unit_type": "METHOD_UNIT"} }	member is None

2.3 ResponseParser

Module: app/utils/response_parser.py

Purpose: Validate XML extraction, JSON parsing, Python-to-JSON keyword conversion, and repetition detection in LLM outputs.

Mock strategy: Pure functions. No mocking. Input: raw LLM output strings.

ID	Type	Scenario	Input	Expected
TC-RP-001	Positive	extract_xml extracts content between XML tags	"<plan>Flatten conditional</plan>"	"Flatten conditional"
TC-RP-002	Positive	extract_xml strips <think> blocks before extraction	"<think>reasoning</think> <code>int x;</code>"	"int x;"
TC-RP-003	Positive	extract_xml validates Java plausibility (semicolons)	"<code>int x = 1;</code>"	"int x = 1;"
TC-RP-004	Positive	extract_xml validates Java plausibility (braces)	"<code>class A { }</code>"	"class A { }"

TC-RP-005	Negative	extract_xml rejects non-Java code tag content	"<code>hello world</code>"	None
TC-RP-006	Edge	extract_xml missing tag returns None	"<other>text</other>"	None
TC-RP-007	Positive	extract_json parses JSON to Pydantic model	'{"refactor_category": "CONTROL_FLOW", "specific_intent": "FLATTEN_CONDITIONAL", "scope_anchor": {"class": "A", "unit_type": "METHOD_UNIT"}}'	Returns IntentPacket instance
TC-RP-008	Positive	extract_json extracts from markdown code fence	` ``json\n{"a":1}\n`` `	{"a": 1}
TC-RP-009	Positive	extract_json fixes trailing comma	'{"a": 1, "b": 2,}'	Parses to {"a": 1, "b": 2}
TC-RP-010	Positive	extract_json fixes Python None → null	'{"member": None}'	{"member": null}
TC-RP-011	Positive	extract_json fixes Python True → true	'{"valid": True}'	{"valid": true}
TC-RP-012	Positive	extract_json fixes Python False → false	'{"valid": False}'	{"valid": false}
TC-RP-013	Edge	Brace inside string value doesn't break depth counting	'{"key": "just {opening", "other": true}'	Parses correctly
TC-RP-014	Edge	Trailing comma inside string value preserved	'{"msg": "ends with,}", "list": [1, 2],}'	"msg": "ends with,}", "list": [1, 2]
TC-RP-015	Edge	Python keywords inside string values not replaced	'{"class": "set to None, True or False}"'	String preserved verbatim
TC-RP-016	Negative	Invalid JSON returns None	"{not valid json"	None
TC-RP-017	Edge	Deeply nested JSON (5 levels)	{"a":{"b":{"c":{"d":{"e":1}}}}}	Parsed fully
TC-RP-018	Positive	detect_repetition detects LLM output loop	Text with 3+ repeating trailing patterns	True
TC-RP-019	Positive	detect_repetition no false positive on normal text	"Varied content with different phrases"	False
TC-RP-020	Edge	detect_repetition handles empty string	""	False

2.4 ASTMatcher

Module: app/utils/ast_matcher.py

Purpose: Validate class/method finding, brace-matched text extraction, and mutation enrichment for constants, fields, method modifications, and symbol renames.

Mock strategy: Pure functions. Java source code + AST mutations as input.

ID	Type	Scenario	Input	Expected
TC-AM-001	Positive	<code>_find_class_declaration_line</code> finds class line	"class A { at line 5	Returns 5
TC-AM-002	Positive	<code>_find_class_declaration_line</code> handles enum/interface	"interface B { at line 2	Returns 2
TC-AM-003	Positive	<code>_find_method_body</code> finds method by name	Code with method <code>calculate()</code>	Returns AST node for <code>calculate</code>
TC-AM-004	Edge	<code>_find_method_body</code> returns None for missing method	Method <code>missing()</code> not in code	Returns None
TC-AM-005	Positive	<code>_extract_method_text</code> braces-matched extraction	Method with nested <code>if/else</code> blocks	Full method text
TC-AM-006	Edge	<code>_extract_method_text</code> with deeply nested braces	Method with 5 levels of nested <code>{}</code>	Correct text boundary
TC-AM-007	Positive	<code>_enrich_add_constant</code> sets <code>insert_after</code> to class decl	Mutation <code>ADD_CONSTANT</code> , target <code>MAX_SIZE</code>	<code>insert_after =</code> class declaration line
TC-AM-008	Positive	<code>_enrich_add_field</code> sets <code>insert_after</code>	Mutation <code>ADD_FIELD</code> , target <code>name</code>	<code>insert_after</code> set correctly
TC-AM-009	Positive	<code>_enrich_modify_method</code> creates <code>find/replace</code>	<code>MODIFY_METHOD</code> with sibling <code>ADD_CONSTANT</code>	<code>replace</code> contains constant references
TC-AM-010	Positive	<code>_enrich_rename_symbol</code> creates <code>find/replace</code> from <code>body_abstract</code>	<code>RENAME_SYMBOL</code> , <code>old=oldName</code> , <code>new=newName</code>	All occurrences replaced
TC-AM-011	Positive	<code>enrich_mutations</code> dispatches by <code>MutationAction</code>	Array with 3 different mutation actions	Each dispatched to correct handler
TC-AM-012	Edge	<code>enrich_mutations</code> handles empty mutation array	<code>[]</code>	Returns empty list

2.5 Validator

Module: app/modules/validator/__init__.py

Purpose: Validate Java syntax checking, complexity computation, intent-specific verification, boundary leak detection, and graceful verifier crash handling.

Mock strategy: No mocking. Real javalang and lizard libraries.

ID	Type	Scenario	Input	Expected
TC-VL-001	Positive	check_syntax validates well-formed Java class	"class A { void m() { int x = 1; } }"	is_valid=True, error=""
TC-VL-002	Positive	check_syntax wraps bare statements in class wrapper	"int x = 1;" (statement-level)	is_valid=True
TC-VL-003	Positive	check_syntax wraps bare method in class wrapper	"void m() { return; }" (method-level)	is_valid=True
TC-VL-004	Negative	check_syntax rejects malformed Java (unclosed brace)	"class A { void m() { "	is_valid=False, error non-empty
TC-VL-005	Edge	check_syntax handles empty input	""	is_valid=False
TC-VL-006	Edge	check_syntax handles whitespace-only input	" \n "	is_valid=False
TC-VL-007	Positive	get_complexity correct CC for method with conditionals	"class A { void m() { if(x) { } if(y) { } } }"	CC = 3
TC-VL-008	Positive	get_complexity returns 1 for class with no methods	"class A { int x; }"	CC = 1
TC-VL-009	Positive	get_method_complexity finds specific method by name	Class with methods setup(), calculate(), teardown	Returns CC for calculate only
TC-VL-010	Negative	get_method_complexity returns None for missing method	Method name "missing" not in class	None
TC-VL-011	Positive	has_structural_change detects structural difference	Original vs refactored with different AST shape	True
TC-VL-012	Positive	has_structural_change confirms identical structure	Code with formatting differences only (whitespace)	False
TC-VL-013	Positive	verify_intent FLATTEN_CONDITIONAL detects nesting decrease	Nested if(a){ if(b){ } } → early return	Passes verification
TC-VL-014	Positive	verify_intent EXTRACT_METHOD detects method count increase	1 method → 2 methods (extracted body)	Passes verification
TC-VL-015	Negative	verify_intent FLATTEN_CONDITIONAL fails when unchanged	Original and refactored are identical	Failure finding
TC-	Negative	verify_intent	Flag done declared but never	Failure finding

VL-016	e	REMOVE_CONTROL_FL AG fails when flag remains	removed	
TC- VL-017	Positive	verify_boundary confirms non-target methods unchanged	Target calculate() changed, setup() unchanged	No violation
TC- VL-018	Negative	verify_boundary detects leak into non-target method	Target calculate() changed, setup() also changed	Boundary violation
TC- VL-019	Positive	verify_complexity STRICT rule requires CC decrease	CC 5 → CC 3	Pass
TC- VL-020	Negative	verify_complexity STRICT rule rejects CC increase	CC 3 → CC 5	Fail
TC- VL-021	Positive	verify_complexity LOOSEMED rule allows CC to stay same	CC 5 → CC 5	Pass
TC- VL-022	Edge	Verifier crash returns graceful ValidationFinding	Verifier raises AttributeError	Returns TIER_2_C finding

2.6 AgentService

Module: app/modules/agent/__init__.py

Purpose: Validate LLM lifecycle: model loading/unloading with GPU layer offloading, grammar-constrained generation, token counting, and interrupt handling.

Mock strategy: Patch llama_cpp.Llama constructor. Mock generate() responses.

ID	Type	Scenario	Input	Expected
TC- AS-001	Positive	generate returns structured response (dict)	Messages list, valid response model	Returns dict with expected keys
TC- AS-002	Edge	generate handles empty messages list	Empty list	Returns gracefully (empty or defaults)
TC- AS-003	Positive	stop triggers generation halt	Call stop() during generate()	InterruptedError raised
TC- AS-004	Positive	load passes correct GPU layers to Llama	Config with layers=36	Llama called with n_gpu_layers=36
TC- AS-005	Positive	unload releases VRAM (calls free + gc)	Model loaded, then unload()	Llama.free() called, gc.collect() called
TC- AS-006	Positive	swap reloads only when config differs	Same config twice	Second load skipped
TC- AS-007	Edge	generate truncates output exceeding max_tokens	Set max_tokens=10, model returns 100 tokens	Output truncated to ~10 tokens
TC- AS-008	Positive	_count_tokens extracts from usage data	Chunk with usage.completion_tokens=42	Returns 42
TC- AS-009	Edge	_count_tokens fallback when no usage data	Empty chunks, content length 100	Returns 25 (len//4)
TC-	Positive	generate applies GBNF	Response model with	Grammar regex

AS-010	e	grammar constraint	required fields	passed to Llama
--------	---	--------------------	-----------------	-----------------

2.7 ClientConnection

Module: app/modules/connection/__init__.py

Purpose: Validate WebSocket message sending (status, result, halt, insights), heartbeat ping/pong mechanics, stale connection detection, and notification suppression.

Mock strategy: Mock WebSocket and database. Full class instantiation.

ID	Type	Scenario	Input	Expected
TC-CC-001	Positive	send_status sends formatted status message via WebSocket	Role=Planner, content="Planning...", phase=2	send_json called with correct message shape
TC-CC-002	Positive	send_result includes model names in result	planner="Model-A", generator="Model-B", judge="Model-C"	send_json with all three model fields
TC-CC-003	Positive	send_halt_notification sends halt message	No input	send_json called with type: "status", content contains "halted"
TC-CC-004	Positive	start_heartbeat sends ping at 15s interval	Heartbeat started	send_json called with type: "ping" within 15s
TC-CC-005	Positive	handle_pong resets missed pong counter	Counter at 1, call handle_pong()	missed_pongs = 0
TC-CC-006	Positive	is_stale true after 2+ missed pongs	Set missed_pongs=2	is_stale = True
TC-CC-007	Edge	is_stale false when within threshold	Set missed_pongs=1	is_stale = False
TC-CC-008	Negative	send_status skipped when client is stale	is_stale=True, call send_status()	send_json not called

2.8 DatabaseManager

Module: app/modules/context/__init__.py

Purpose: Validate SQLite CRUD operations for session lifecycle, zombie cleanup, cascading deletes, and Tenacity retry logic on OperationalError.

Mock strategy: Real in-memory SQLite via Peewee. No mocks. Each test uses fresh DB.

ID	Type	Scenario	Input	Expected
TC-DB-001	Positive	create_session inserts row with correct fields	session_id="s1", instruction="refactor", code="class A {}"	Row created, fields match
TC-DB-002	Positive	complete_session updates status + code + metrics	session_id="s1", exit="SUCCESS", code="class B {}"	Row updated, exit_status = "SUCCESS"
TC-	Positive	log_status creates	session_id="s1",	Log created with

DB-003	e	OrchestrationLog entry	role="Planner", content="planning"	backref to session
TC-DB-004	Positive	mark_as_halted sets status to Halted	session_id="s1", call mark_as_halted()	status = "Halted"
TC-DB-005	Positive	get_history returns stubs ordered by date desc	Insert 3 sessions at different times	Returns 3 stubs, newest first
TC-DB-006	Positive	get_history_by_id returns full detail with nested logs	Create session + 2 log entries	Returns session + 2 logs
TC-DB-007	Negative	get_history_by_id returns None for non-existent ID	ID "nonexistent"	None
TC-DB-008	Positive	rename_session updates title	session_id="s1", new title="My Refactor"	title = "My Refactor"
TC-DB-009	Positive	cleanup_zombie_sessions marks old Processing as Zombie	Session with status "Processing", older than threshold	Status changed to "Zombie"
TC-DB-010	Positive	cleanup_halted_sessions deletes old halted sessions	Session with status "Halted", older than threshold	Row deleted
TC-DB-011	Positive	delete_history_by_id cascading delete	Session + 2 logs, call delete	Session gone, logs gone
TC-DB-012	Edge	Retry logic on OperationalError	Mock 2 failures, then success	Database operation eventually succeeds

2.9 MessageRouter

Module: app/modules/connection/router.py

Purpose: Validate WebSocket message dispatching: pong heartbeat, session reconnect, single/multi refactor launch, halt propagation, and invalid payload rejection.

Mock strategy: Mock AgentService, ClientConnection, and task factory functions.

ID	Type	Scenario	Input	Expected
TC-MR-001	Positive	dispatch routes pong to client.handle_pong()	{"type": "pong"}	client.handle_pong() called
TC-MR-002	Positive	dispatch routes reconnect to reconnect handler	{"type": "reconnect", "session_id": "s1"}	Reconnect handler called with "s1"
TC-MR-003	Positive	dispatch validates + launches single refactor	{"type": "single", "code": "class A {}", "user_instruction": "refactor"}	run_single task factory called
TC-MR-004	Positive	dispatch validates + launches multi-agent refactor	{"type": "multi", "code": "class A {}", "user_instruction": "refactor"}	run_multi task factory called
TC-MR-005	Positive	dispatch routes halt to agent stop + task cancel	{"type": "halt"}	agent.stop() called, active tasks cancelled

TC-MR-006	Negative	dispatch rejects invalid refactor request	{"type": "multi", "code": ""}	Error response sent, no task launched
TC-MR-007	Negative	dispatch rejects invalid JSON	malformed (not parseable)	Error response sent
TC-MR-008	Edge	dispatch ignores unknown message type	{"type": "unknown"}	No action, no error

2.10 Orchestrator

Module: app/modules/orchestrator/__init__.py

Purpose: Validate orchestration state management (feedback ring buffer, strategy iteration), notification routing with model names, and main pipeline execution control flow (phase sequencing, retry, abort).

Mock strategy: Mock AgentService, ClientConnection, DatabaseManager. Test `OrchestrationState` methods directly.

ID	Type	Scenario	Input	Expected
TC-OR-001	Positive	OrchestrationState initializes with valid defaults	session_id="s1", base_code="class A {}", working_code="class A {}", user_instruction="refactor"	State has strategy_iter=1, empty feedback, exit_status="PROCESSING"
TC-OR-002	Positive	add_feedback appends to ring buffer	State with 2 feedback entries, add a 3rd	feedback has 3 entries
TC-OR-003	Edge	add_feedback caps ring buffer at 3	State with 3 entries, add a 4th	feedback has 3 entries, oldest evicted
TC-OR-004	Positive	extend_feedback batch-adds with cap maintained	State with 2 entries, extend with 3 more	feedback has 3 entries (most recent retained)
TC-OR-005	Positive	_notify sends status to WebSocket client	Role=Planner, content="Analyzing...", phase=2	client.send_status() called with correct args
TC-OR-006	Positive	_notify includes model names when phase is strategy	phase=2, planner_model="Model-A"	send_status called with planner_model="Model-A"
TC-OR-007	Edge	_notify skips send when client is stale	client.is_stale=True, call _notify()	send_status not called, DB log still created
TC-OR-008	Positive	execute_orchestration runs phases 1→2→3→4→5→6 in sequence	Valid state, all phases pass	Phase methods called in order, returns ExitStatus.SUCCESS
TC-OR-009	Edge	execute_orchestration retries strategy on Phase 4 failure	Intent verification fails, retry < max	Phase 2 re-invoked with feedback
TC-OR-010	Positive	execute_orchestration exits on max strategy retries	Intent verification fails 3 times	Returns ExitStatus.ABORT_ST

				RATEGY
TC-OR-011	Positive	execute_orchestration halts gracefully on interrupt	InterruptedError raised during Phase 3	Returns ExitStatus.ABORT_ST RATEGY, cleanup called
TC-OR-012	Edge	execute_orchestration creates _notify calls for each phase transition	Full pipeline	DB has 1 log per phase _notify call

2.11 Phase 2 — Strategy

Module: app/modules/orchestrator/phases/phase2_strategy.py

Purpose: Validate intent classification, architecture analysis, synthesis with retry on invalid plans, ASTMatcher enrichment, mutation deduplication, and feedback translation.

Mock strategy: Mock AgentService.generate responses. Test _classify, _analyze, _synthesize, _enrich, _deduplicate in isolation.

ID	Type	Scenario	Input	Expected
TC-P2-001	Positive	_classify returns IntentPacket from LLM response	Mock generate returns IntentClassifierResponse with FLATTEN_CONDITIONAL	state.intent_packet .specific_intent == FLATTEN_CONDITIONAL
TC-P2-002	Negative	_classify handles LLM returning invalid intent	Mock returns invalid category	Returns fallback/extraction error, no crash
TC-P2-003	Positive	_analyze stores ArchitectAnalysisResponse	Mock returns analysis with primary targets	state.architect_analysis populated
TC-P2-004	Edge	_analyze handles bad JSON from LLM gracefully	Mock returns unparseable response	state.architect_analysis = {}, orchestration continues
TC-P2-005	Positive	_synthesize returns ASTArchitectResponse with mutations	Mock returns plan with 3 mutations	state.active_plan has 3 ast_mutations
TC-P2-006	Edge	_synthesize retries on invalid plan (too many mutations)	Mock returns 7 mutations (max is 5)	Retries synthesis with feedback for fewer mutations
TC-P2-007	Positive	_synthesize gives up after max retries	Mutations always > 5 across all retries	Returns, falls back to last valid plan or empty
TC-P2-008	Positive	_enrich calls ASTMatcher to fill find/replace	State has MutationAction.MODIFY_METHOD	ASTMatcher called, mutations have find/replace fields
TC-P2-009	Positive	_deduplicate removes duplicate (action, target) pairs	2 mutations with same MODIFY_METHOD on calculate	1 remains, others removed
TC-P2-010	Positive	_deduplicate preserves unique pairs	3 mutations, different targets	All 3 remain

TC-P2-011	Positive	<code>_translate_feedback</code> maps TIER_1 to syntax hint	ValidationFinding with TIER_1_SYNTAX	Feedback hint contains "syntax" instruction for LLM
TC-P2-012	Positive	<code>_translate_feedback</code> maps TIER_2_C to intent hint	TIER_2_C_INTENT_MATH	Feedback hint contains specific intent guidance

2.12 Phase 3 — Execution

Module: `app/modules/orchestrator/phases/phase3_execution.py`

Purpose: Validate generator output repair (throws, null-check, public stripping), multi-temperature best-CC selection, and sequential mutation with syntax healing and boundary break halting.

Mock strategy: Mock `AgentService.generate`. Test `repair_generator_output` as pure function.

ID	Type	Scenario	Input	Expected
TC-P3-001	Positive	<code>repair_generator_output</code> strips throws Exception	Generated adds throws Exception on method	Original method signature restored
TC-P3-002	Positive	<code>repair_generator_output</code> strips null-check wrappers	Generated wraps body in <code>if (x != null)</code>	Null check removed, original body preserved
TC-P3-003	Positive	<code>repair_generator_output</code> strips extra <code>public</code> modifier	Generated makes method <code>public</code> when original was <code>package-private</code>	Visibility restored to original
TC-P3-004	Edge	<code>repair_generator_output</code> leaves valid code unchanged	Generated already matches original style	No changes made
TC-P3-005	Positive	<code>run_single</code> generates code at 3 temperatures, picks best CC	Mock returns 3 outputs with CC 5, 3, 7	Output with CC=3 selected
TC-P3-006	Edge	<code>run_single</code> uses fallback on all temperatures failing	All 3 outputs have syntax errors	Returns best-effort or fallback message
TC-P3-007	Positive	<code>run_sequential</code> applies one mutation at a time	2 mutations, both pass validation	<code>state.working_code</code> reflects both mutations applied sequentially
TC-P3-008	Edge	<code>run_sequential</code> retries on syntax error (healing)	Mutation produces syntax error, <code>retry < max</code>	Retry generates corrected code
TC-P3-009	Edge	<code>run_sequential</code> halts on boundary break	Mutation leaks into non-target method	Sequential halts, remaining mutations skipped
TC-P3-010	Edge	<code>run_sequential</code> gives up after max syntax heal attempts	Syntax error persists after all retries	Returns to caller, Phase 4 triggers replan

2.13 Phase 4 — Validation

Module: app/modules/orchestrator/phases/phase4_validation.py

Purpose: Validate the full validation chain (syntax, complexity, boundary, intent) and correct routing to Phase 3 for healing or Phase 2 for replan based on failure tier.

Mock strategy: Validator used real (javalang, lizard). Mock state to simulate pass/fail.

ID	Type	Scenario	Input / Arrange	Expected
TC-P4-001	Positive	run passes all validation checks (syntax, CC, boundary, intent)	State has valid working code, CC decreased, no boundary leak, intent matched	Returns route=ACCEPT, validation_faults=0
TC-P4-002	Negative	run routes to Phase 3 when syntax check fails	Working code is invalid Java	Routes to Phase 3 (syntax healing)
TC-P4-003	Negative	run routes to Phase 2 when intent verification fails	Code valid but refactoring doesn't match intent	Routes to Phase 2 (replan)
TC-P4-004	Positive	run passes CC check with STRICT rule	Original CC=5, working CC=3	CC check passes
TC-P4-005	Negative	run fails CC check with STRICT rule	Original CC=3, working CC=5	CC check fails, fault added
TC-P4-006	Positive	run passes boundary check	Only target method changed	No boundary faults
TC-P4-007	Negative	run fails boundary check (leak detected)	Non-target method also changed	Boundary fault added
TC-P4-008	Edge	run reports all faults regardless of route	Multiple failures across syntax/CC/boundary/intent	All faults reported in state.checks, highest-priority route selected

2.14 Phase 5 — Adjudication

Module: app/modules/orchestrator/phases/phase5_adjudication.py

Purpose: Validate judge verdict routing (ACCEPT to Phase 6, REVISE to Phase 2), hallucination overrides (identical code, logic drift), and structured issue extraction.

Mock strategy: Mock AgentService.generate. Test judge verdict routing.

ID	Type	Scenario	Input / Arrange	Expected
TC-P5-001	Positive	run judge returns ACCEPT, proceeds to Phase 6	Mock returns { "verdict": "ACCEPT" }	Phase 6 called, pipeline continues
TC-P5-002	Positive	run judge returns REVISE, returns to Phase 2	Mock returns { "verdict": "REVISE" }	Routes to Phase 2, strategy_iter increments
TC-P5-003	Positive	run detects IDENTICAL_CODE hallucination, overrides to REVISE	Judge analysis shows original and refactored are identical	Overrides to REVISE regardless of LLM verdict
TC-P5-	Positive	run detects LOGIC_DRIFT	Judge analysis shows logic	Overrides to

004	e	hallucination, overrides to REVISE	divergence from original	REVISE
TC-P5-005	Edge	run handles missing/empty judge response	Mock returns None	Assumes REVISE, logs error, returns to Phase 2
TC-P5-006	Edge	run extracts structured issues from judge response	Mock returns { "verdict": "REVISE", "issues": [...] }	Issues parsed and stored in state.judge_issues

2.15 Phase 6 — Finalization

Module: app/modules/orchestrator/phases/phase6_finalization.py

Purpose: Validate outer wrapper stripping, result transmission with metrics, LLM-generated insights, database persistence, and graceful insight-generation failure handling.

Mock strategy: Mock AgentService.generate for insights. Mock DB for persistence.

ID	Type	Scenario	Input / Arrange	Expected
TC-P6-001	Positive	run strips outer wrapper from generated code	Generated code wrapped in fabricated class, original had none	Wrapper removed, only method body retained
TC-P6-002	Positive	run sends result with correct code and metrics	Valid state with working code and metrics	client.send_result() called with code, CC values, performance data
TC-P6-003	Positive	run generates insights from LLM	Mock returns RefactorInsightsResponse with 2 insights	Insights stored in state.orchestration_result
TC-P6-004	Edge	run handles LLM insight generation failure gracefully	Mock returns unparseable response	Insights skipped, pipeline does not crash
TC-P6-005	Positive	run persists session to database	Valid state, all fields populated	db.create_session() and db.complete_session() called
TC-P6-006	Positive	run attaches performance metrics to result	PerformanceTracker metrics provided	send_result called with performance_metrics matching tracker output

2.16 Formatters

Module: app/utils/formatters.py

Purpose: Validate conversion of JSON mutation plans into human-readable linear text instructions for the generator, including constant references and logical ordering.

Mock strategy: Pure function. Input: JSON mutation plan + base code.

ID	Type	Scenario	Input	Expected
TC-FMT-001	Positive	format_plan_for_generator converts single mutation to text	Plan with 1 MODIFY_METHOD on calculate	Linear text: "Modify method calculate\n Action: MODIFY_METHOD\n Details: ..."
TC-FMT-002	Positive	format_plan_for_generator handles ADD_CONSTANT with value	ADD_CONSTANT with MAX_SIZE=100	Text includes "Add constant MAX_SIZE = 100"
TC-FMT-003	Positive	format_plan_for_generator includes constant references in MODIFY	MODIFY_METHOD with sibling ADD_CONSTANT	Text references the new constant name
TC-FMT-004	Positive	format_plan_for_generator orders mutations logically	Mix of RENAME, ADD_CONSTANT, MODIFY_METHOD	Output lists RENAME first, then ADD, then MODIFY
TC-FMT-005	Edge	format_plan_for_generator handles empty mutation array	{"ast_mutations": []}	Returns empty string or "No changes"

2.17 Code Utils

Module: app/utils/code_utils.py

Purpose: Validate outer wrapper stripping logic and mutation ordering (rename-first, then add, then modify/remove, stable sort within groups).

Mock strategy: Pure functions. Input: Java code strings + mutation arrays.

ID	Type	Scenario	Input	Expected
TC-CU-001	Positive	strip_outer_wrapper removes fabricated class wrapper	Generated code wrapped in public class TempWrapper { ... }, base_code is bare method	Only inner content returned
TC-CU-002	Edge	strip_outer_wrapper preserves code when no wrapper needed	Generated code already matches expected structure	Unchanged (identity)
TC-CU-003	Edge	strip_outer_wrapper handles class body only (no outer)	Base code is already a class	Returns input unchanged
TC-CU-004	Positive	order_mutations places RENAME_SYMBOL first	Array: [MODIFY_METHOD, RENAME_SYMBOL, ADD_FIELD]	RENAME_SYMBOL first, then ADD_FIELD, then MODIFY_METHOD
TC-	Positive	order_mutations places	Array: [REMOVE_METHOD,	ADD_DECLARATI

CU-005	e	ADD_* before MODIFY/REMOVE	ADD_DECLARATION, MODIFY_METHOD]	ON first, then MODIFY, then REMOVE
TC-CU-006	Edge	order_mutations preserves relative order within same priority group	2 MODIFY_METHOD mutations in arbitrary order	Same relative order maintained (stable sort)

Section 3: Backend Integration Tests

Module: FastAPI application (app/main.py)

Purpose: Validate end-to-end REST API and WebSocket endpoint behavior including health check, history CRUD, and real-time message exchange (heartbeat, halt acknowledgment, error rejection).

Approach: FastAPI `TestClient` with real in-memory SQLite database. WebSocket tested via ASGI transport.

ID	Type	Scenario	Input	Expected
TC-IT-001	Positive	GET /health returns 200 with timestamp	—	Status 200, body has {"status": "ok", "timestamp": "..."} }
TC-IT-002	Positive	GET /api/history returns history stub list	One session created	Status 200, body has array with 1 entry
TC-IT-003	Positive	GET /api/history/:id returns full detail	Session with code + logs	Status 200, body has logs, original_code, refactored_code
TC-IT-004	Negative	GET /api/history/:id returns 404 for nonexistent	ID "invalid"	Status 404
TC-IT-005	Positive	PATCH /api/history/:id renames session	{"title": "New Title"}	Status 200, double-check via GET
TC-IT-006	Negative	PATCH /api/history/:id rejects empty title	{"title": ""}	Status 422
TC-IT-007	Positive	DELETE /api/history/:id deletes session	Session exists	Status 200, GET after returns 404
TC-IT-008	Negative	DELETE /api/history/:id returns 404 for missing	ID "invalid"	Status 404
TC-IT-009	Positive	WebSocket /ws sends connection_id on connect	WS connect	Receives message with type: "connection_id" and id
TC-IT-010	Positive	WebSocket /ws sends periodic heartbeat ping	WS connected, wait 5 seconds	Receives message with type: "ping"
TC-IT-011	Positive	WebSocket /ws sends halt_acknowledged on halt	Send {"type": "halt"}	Receives {"type": "halt_acknowledged"}

				}
TC-IT-012	Negative	WebSocket /ws rejects malformed JSON	Send not json	Receives error message with type: "error"

Section 4: Frontend Unit Tests

4.1 lib/utils — cn()

Module: `src/lib/utils.ts`

Purpose: Validate Tailwind CSS class merging, falsy filtering, conditional object syntax, and tailwind-merge conflict resolution.

Mock strategy: Pure function. No mocking.

ID	Type	Scenario	Input	Expected
TC-CN-001	Positive	Merges multiple class name strings	<code>cn("foo", "bar")</code>	"foo bar"
TC-CN-002	Positive	Filters falsy values	<code>cn("foo", false, "bar", undefined, null, "baz")</code>	"foo bar baz"
TC-CN-003	Positive	Handles conditional object syntax	<code>cn("base", { active: true, disabled: false })</code>	"base active"
TC-CN-004	Positive	Resolves Tailwind conflicts via <code>twMerge</code>	<code>cn("px-4", "px-2")</code>	"px-2"
TC-CN-005	Edge	Returns empty string for no args	<code>cn()</code>	""
TC-CN-006	Positive	Handles array arguments	<code>cn(["foo", "bar"], "baz")</code>	"foo bar baz"

4.2 lib/constants

Module: `src/lib/constants.ts`

Purpose: Validate constant shape, role visuals completeness, and default fallback structure.

Mock strategy: Read-only constant values. No mocking.

ID	Type	Scenario	Expected
TC-CNST-001	Positive	INITIAL_SOURCE is empty string	""
TC-CNST-002	Positive	EMPTY_ORCHESTRATION_RESULT has expected shape	Contains metrics: [], summary: "", diffHighlights: { added: [], removed: [] }
TC-CNST-003	Positive	ROLE_VISUALS has entries for all 6 roles	Object has keys for Planner, Generator, Validator, Judge, System, Monolith
TC-CNST-004	Positive	Each ROLE_VISUALS entry has step, icon, colorClass	All 18 fields present and typed correctly

4			
TC-CNST-005	Positive	DEFAULT_ROLE_VISUALS has fallback values	{ step: 1, icon: "Cpu", colorClass: "text-jb-accent" }

4.3 lib/parseStatusInfo

Module: src/lib/parseStatusInfo.ts

Purpose: Validate text parsing for phase numbers, strategy iterations, retry info, validation faults, judge decisions, intent details, and mutation plans with JSON/markdown fallbacks.

Mock strategy: Pure functions. Input: raw status string content.

ID	Type	Scenario	Input	Expected
TC-PS-001	Positive	parsePhaseNumber detects "Ph1" pattern	"Ph1: Analyzing baseline..."	1
TC-PS-002	Positive	parsePhaseNumber detects "Baseline" keyword	"Starting Baseline phase"	1
TC-PS-003	Positive	parsePhaseNumber detects "Strategy" keyword	"Strategy phase analyzing..."	2
TC-PS-004	Positive	parseStrategyIteration extracts iteration number	"Strategy Iter 2"	2
TC-PS-005	Positive	parseRetryInfo detects syntax heal attempts	"Syntax heal attempt 2/3"	{ current: 2, max: 3, type: "syntax_heal" }
TC-PS-006	Positive	parseRetryInfo detects sequential mutation retry	"Retry mutation 1/5"	{ current: 1, max: 5, type: "sequential" }
TC-PS-007	Positive	parseValidationFaults extracts fault count	"Total Faults: 3"	3
TC-PS-008	Positive	parseJudgeDecision detects ACCEPT verdict	"Decision: ACCEPT"	"ACCEPT"
TC-PS-009	Positive	parseJudgeDecision detects REVISE verdict	"Verdict: REVISE"	"REVISE"
TC-PS-010	Positive	parseIntentDetail extracts from embedded JSON	Content with {"category": "CONTROL_FLOW", ...}	Returns IntentDetail object
TC-PS-011	Edge	parseIntentDetail falls back to markdown regex	Content with Category: CONTROL_FLOW\n Intent: FLATTEN_CONDITIONAL	Returns IntentDetail via regex
TC-PS-012	Positive	parseMutationPlan extracts from JSON array	Content with "ast_mutations": [...]	Returns MutationItem[]
TC-PS-013	Edge	parseMutationPlan falls back to markdown list	Content with - MODIFY_METHOD on \calculate\`	Returns MutationItem[] via regex
TC-PS-014	Positive	parsePhaseAction extracts action description	"Ph2: Analyzing code structure"	"Analyzing code structure"

4.4 lib/formatStatusContent

Module: `src/lib/formatStatusContent.ts`

Purpose: Validate extraction of JSON blocks, key-value tags, markdown formatting, first-line summary derivation, and recursive flattening of nested objects.

Mock strategy: Pure functions. Input: raw status strings.

ID	Type	Scenario	Input	Expected
TC-FS-001	Positive	Extracts JSON blocks into structured format	Content with <code>\\json ... \\</code> block	summary contains extracted values, tags populated
TC-FS-002	Positive	Detects standalone JSON objects	Content starting with <code>{</code>	Parsed and flattened
TC-FS-003	Positive	Extracts <code>**Key:** Value</code> tags	<code>**Category:** CONTROL_FLOW</code>	tags includes <code>{ label: "Category", value: "CONTROL_FLOW" }</code>
TC-FS-004	Positive	Extracts bullet-tag items	<code>- **Intent:** FLATTEN_CONDITIONAL</code>	tags includes parsed item
TC-FS-005	Positive	Returns summary from first line	<code>"Starting analysis.\nMore details here"</code>	summary: <code>"Starting analysis."</code>
TC-FS-006	Positive	Flattens nested JSON recursively	Nested object with 3 levels	All leaf values accessible
TC-FS-007	Edge	Handles empty input	<code>""</code>	summary: <code>""</code> , empty tags
TC-FS-008	Edge	Handles input with no recognizable patterns	<code>"plain text"</code>	summary: <code>"plain text"</code> , empty tags

4.5 lib/javaFormatter

Module: `src/lib/utils/javaFormatter.ts`

Purpose: Validate Java code formatting: operator spacing normalization, brace merging for `else/catch/finally`, string/comment/generic preservation, placeholder extraction and restoration.

Mock strategy: Pure function. Input: raw Java code strings.

ID	Type	Scenario	Input	Expected
TC-JF-001	Positive	Formats basic class with braces and indentation	<code>"class A { void m() { } }"</code>	Correctly indented multi-line output
TC-JF-002	Positive	Preserves string literals during formatting	Code with <code>"select * from table"</code>	String unchanged
TC-JF-003	Positive	Preserves comments during formatting	Code with <code>// comment</code> and <code>/* block */</code>	Comments preserved
TC-JF-004	Positive	Handles generic type parameters	<code>List<Map<String, Integer>></code>	Generics formatted correctly
TC-JF-005	Positive	Normalizes spaces around operators	<code>a=b+c</code>	<code>a = b + c</code>
TC-JF-006	Positive	Merges <code>else/catch/finally</code> to previous closing brace	<code>} \n else {</code>	<code>} else {</code>

TC-JF-007	Edge	Strips consecutive blank lines	3 blank lines in a row	1 blank line
TC-JF-008	Edge	Restores placeholders after formatting	Contains extracted strings/comments	Original values reappear
TC-JF-009	Edge	Handles empty string	""	""
TC-JF-010	Positive	Handles long method with mixed constructs	Method with loops, conditionals, try-catch, lambdas	Correctly indented

4.6 lib/indentation

Module: `src/lib/indentation.ts`

Purpose: Validate smart Enter (auto-indent after line, auto-close brace without duplication), Tab/Shift-Tab indent/outdent, and closing brace alignment to matching opening brace.

Mock strategy: Pure functions. Input: code string + cursor position.

ID	Type	Scenario	Input	Expected
TC-IN-001	Positive	<code>handleEnterKey</code> adds indent after line	Line <code>if (x) {</code> at end	New line with indent
TC-IN-002	Positive	<code>handleEnterKey</code> auto-closes brace after <code>{</code>	Line <code>main() {</code> at end	Inserts <code>\n \n}</code> with cursor at middle
TC-IN-003	Edge	<code>handleEnterKey</code> no double close brace	Line already has <code>}</code> closing brace	Only indents, no extra <code>}</code>
TC-IN-004	Positive	<code>handleTabKey</code> indents current line by 4 spaces	Line <code>foo();</code> with cursor at start	Line becomes <code>foo();</code>
TC-IN-005	Positive	<code>handleTabKey</code> indents selected block	3 lines selected	All 3 lines indented 4 spaces
TC-IN-006	Positive	<code>handleShiftTab</code> outdents line	Line <code>foo();</code>	<code>foo();</code>
TC-IN-007	Positive	<code>handleClosingBrace</code> adjusts indent to match opening brace	<code>if(x) {\n }</code> (cursor after <code>)</code>	<code>if(x) {\n }</code> (brace at same level as <code>if</code>)
TC-IN-008	Edge	<code>handleClosingBrace</code> returns null when no adjustment needed	<code>}</code> already correctly aligned	null

4.7 lib/buildMetrics

Module: `src/lib/utils/buildMetrics.ts`

Purpose: Validate construction of cyclomatic complexity metrics (direction up/down/neutral), inference time, and GPU utilization/memory metrics from raw performance payloads.

Mock strategy: Pure function. Input: complexity numbers + performance metrics.

ID	Type	Scenario	Input	Expected
TC-BM-001	Positive	Builds complexity metric when complexity decreased	orig=10, refactored=5	Metric with direction: "down", before="10", after="5"
TC-BM-002	Positive	Builds complexity metric when complexity increased	orig=5, refactored=10	Metric with direction: "up"
TC-BM-003	Positive	Builds complexity metric when unchanged	orig=5, refactored=5	Metric with direction: "neutral"
TC-BM-004	Positive	Builds inference time metric	performance has inference_time: 12.5	Metric with after: "12.5s"
TC-BM-005	Positive	Builds GPU metrics when performance provided	performance with avg_gpu_utilization, peak_gpu_memory_used	GPU metrics present
TC-BM-006	Edge	Skips GPU metrics when performance is null/undefined	performance = null or undefined	Only complexity metric returned

4.8 lib/schemas/websocket (Zod)

Module: `src/lib/schemas/websocket.ts`

Purpose: Validate Zod schema validation for all WebSocket message types and discriminated union routing for type-safe message handling.

Mock strategy: Pure Zod schema validation. No mocking.

ID	Type	Scenario	Input	Expected
TC-ZD-001	Positive	StatusMessageSchema validates correct status	{ type: "status", role: "Planner", content: "Analyzing..." }	Passes parse
TC-ZD-002	Negative	StatusMessageSchema rejects missing role	{ type: "status", content: "..." }	Fails parse
TC-ZD-003	Negative	StatusMessageSchema rejects invalid type	{ type: "invalid", role: "Planner", content: "..." }	Fails parse
TC-ZD-004	Positive	ResultMessageSchema validates correct result	{ type: "result", code: "class A {}", exit_status: "SUCCESS" }	Passes parse
TC-ZD-005	Negative	ResultMessageSchema rejects invalid exit_status	{ type: "result", code: "...", exit_status: "INVALID" }	Fails parse
TC-ZD-006	Positive	ErrorMessageSchema validates correct error	{ type: "error", message: "Something went wrong" }	Passes parse

TC-ZD-007	Positive	ServerMessageSchema discriminates by type	Status message, result message, error message	Each matches correct schema in union
TC-ZD-008	Negative	ServerMessageSchema rejects unknown type	{ type: "unknown" }	Fails parse

4.9 store/useChatStore

Module: `src/store/useChatStore.ts`

Purpose: Validate Zustand store operations: session CRUD, API integration (fetch, rename with optimistic update, delete with rollback), session ID migration, and draft management.

Mock strategy: Real Zustand store. Mock `global.fetch` for API calls.

ID	Type	Scenario	Input	Expected
TC-ST-001	Positive	<code>createSession</code> creates session with defaults	<code>createSession("test-1")</code>	Session with title: "New Session", empty <code>sourceCode</code> , empty <code>terminalEntries</code>
TC-ST-002	Positive	<code>createSessionWithInitialPrompt</code> derives title from prompt	<code>prompt = "Flatten conditional in calculate method"</code>	<code>title = first 48 chars of prompt</code>
TC-ST-003	Positive	<code>updateSession</code> partial update merges correctly	Create session, <code>updateSession("test-1", { title: "New Title" })</code>	Title changed, other fields unchanged
TC-ST-004	Positive	<code>updateSession</code> updater function accumulates state	Call <code>updateSession</code> with 2 updater calls that push to <code>terminalEntries</code>	<code>terminalEntries</code> has 2 entries
TC-ST-005	Positive	<code>deleteSession</code> calls DELETE API + removes from sessions	Session exists, fetch mock returns 200	Session removed, fetch called with DELETE
TC-ST-006	Negative	<code>deleteSession</code> rolls back on API failure	Mock fetch returns 500	Session restored, error propagated
TC-ST-007	Positive	<code>renameSession</code> optimistic update on PATCH success	Mock PATCH returns 200	Title updated in store
TC-ST-008	Negative	<code>renameSession</code> rollback on PATCH failure	Mock PATCH returns 500	Title reverted to original
TC-ST-009	Positive	<code>migrateSessionId</code> moves data to new key	Session "old-id" → <code>migrateSessionId("old-id", "new-id")</code>	"old-id" removed, "new-id" has same data
TC-ST-010	Positive	<code>fetchHistory</code> populates sessions from API	Mock GET returns 2 stubs	Store has 2 sessions with correct titles
TC-ST-011	Negative	<code>fetchHistory</code> sets error flag on failure	Mock GET throws	<code>historyLoadError = true</code>
TC-ST-012	Positive	<code>fetchSessionDetails</code> maps logs to terminal entries	Mock GET returns session with 2 log entries	Store has 2 terminal entries

				with correct badges
TC-ST-013	Positive	resetDraftSession clears all draft fields	Draft has values, call resetDraftSession()	Draft back to initial defaults
TC-ST-014	Positive	setOrchestratorStatus updates status	setOrchestratorStatus("connected")	orchestratorStatus = "connected"

4.10 hooks/useOrchestrationSocket

Module: src/hooks/useOrchestrationSocket.tsx

Purpose: Validate WebSocket lifecycle (connect/disconnect), message routing (status, result, insights, error, halt, connection_id), glassbox state updates, and request deduplication.

Mock strategy: Mock WebSocket constructor. Mock useChatStore actions.

ID	Type	Scenario	Input	Expected
TC-WS-001	Positive	connect opens WebSocket to correct URL	connect("session-1")	WebSocket constructed with WS_URL
TC-WS-002	Positive	disconnect closes socket (no auto-reconnect)	Call disconnect()	close() called, reconnect flag = false
TC-WS-003	Positive	Status message updates glassbox state	Receive { type: "status", role: "Planner", content: "Ph2..." }	glassboxState.currentPhase updated
TC-WS-004	Positive	Result message sets refactored output	Receive { type: "result", code: "class B {}", exit_status: "SUCCESS" }	Store session.refactoredOutput updated
TC-WS-005	Positive	Insights message updates orchestration result	Receive { type: "insights", insights: [{ title: "CC", details: "Improved" }] }	orchestrationResult.summary set
TC-WS-006	Positive	Error message creates terminal entry	Receive { type: "error", message: "Failed" }	Store terminalEntries has entry with type: "error"
TC-WS-007	Positive	Halt acknowledged sets appState to idle	Receive { type: "halt_acknowledged" }	Store appState = "idle"
TC-WS-008	Positive	sendRefactorRequest sends multi type message	sendRefactorRequest({ code: "...", user_instruction: "..." })	WebSocket send called with { type: "multi", ... }
TC-WS-009	Positive	Connection ID migrates session ID	Receive { type: "connection_id", id: "ws-123" }	migrateSessionId called with correct IDs
TC-WS-010	Negative	sendRefactorRequest deduplicates by commandId	Same commandId sent twice	send called only once

4.11 components/ErrorMessage

Module: `src/components/layout/ErrorMessage.tsx`

Purpose: Validate React error boundary rendering, error catching from crashing children, custom fallback display, and error state reset on children replacement.

Mock strategy: Spy on `console.error` (React logs caught errors). Test with crashing child.

ID	Type	Scenario	Input	Expected
TC-EB-001	Positive	Renders children when no error	<code><ErrorMessage><div>ok</div></ErrorMessage></code>	Shows "ok"
TC-EB-002	Positive	Catches error from crashing child and shows fallback	Child throws <code>new Error("crash")</code>	Shows fallback UI (not blank page)
TC-EB-003	Positive	Shows custom fallback when provided	<code>fallback={<CustomFallback />}</code> + crashing child	Shows <code><CustomFallback /></code>
TC-EB-004	Edge	Resets error state when children are replaced	Catch error, then re-mount with non-crashing child	Shows new children, error cleared

Section 5: Frontend Integration Tests

Approach: React Testing Library with real store (Zustand) + mocked WebSocket boundary.

5.1 Terminal

Module: `src/components/features/terminal/Terminal.tsx`

Purpose: Validate rendering of all terminal entry types (command, log, system, error, divider), auto-scroll behavior on new entries, and collapsible panel toggle.

Mock strategy: Real store (Zustand) with pre-populated terminal entries. Mock WebSocket boundary.

ID	Type	Scenario	Input / Arrange	Expected
TC-FI-001	Positive	Renders all terminal entry types	Store has entries: command, log, system, error, divider	Each rendered with correct styling and badges
TC-FI-002	Positive	Auto-scrolls to bottom on new entry	Render with entries, add new entry	Scroll container shows latest entry
TC-FI-003	Positive	Collapsible header toggles panel visibility	Click chevron on header	Panel content hides/shows

5.2 ChatWorkspace

Module: `src/components/features/workspace/ChatWorkspace.tsx`

Purpose: Validate panel layout rendering (input, output, terminal), session not-found error banner display, and end-to-end analysis flow via WebSocket request.

Mock strategy: Real store (Zustand) with session data. Mock WebSocket via `useOrchestrationSocket`.

ID	Type	Scenario	Input / Arrange	Expected
TC-FI-004	Positive	Renders all 3 panel areas (input, output, terminal)	Render with valid sessionId	InputPanel, RefactoredOutput, Terminal all in DOM
TC-FI-005	Positive	Shows error banner when session not found	Session ID not in store	Error banner with "not found" message
TC-FI-006	Positive	Start analysis sends refactor request via WebSocket	Click submit, valid code + instruction	<code>sendRefactorRequest</code> called, <code>appState</code> = "analyzing"

Section 6: Test Execution Matrix

Module	Test File	Test Cases	Priority	Dependencies
BACKEND UNIT		170		
OrchestrationConfig	test_config.py	5	High	YAML file
Types & Schemas	test_types_schemas.py	8	High	None
ResponseParser	test_response_parser.py	20	High	None
ASTMatcher	test_ast_matcher.py	12	High	Java source
Validator	test_validator.py	22	Critical	javalang, lizard
AgentService	test_agent_service.py	10	High	llama_cpp (mocked)
ClientConnection	test_connection.py	8	High	WebSocket (mocked)
DatabaseManager	test_context.py	12	Critical	In-memory SQLite
MessageRouter	test_router.py	8	Medium	Agent + Connection
Orchestrator	test_orchestrator.py	12	Critical	Agent, DB, Connection (mocked)
Phase 2 — Strategy	test_phase2_strategy.py	12	Critical	AgentService (mocked)
Phase 3 — Execution	test_phase3_execution.py	10	Critical	AgentService (mocked)
Phase 4 — Validation	test_phase4_validation.py	8	Critical	Validator (real)
Phase 5 — Adjudication	test_phase5_adjudication.py	6	Critical	AgentService (mocked)
Phase 6 — Finalization	test_phase6_finalization.py	6	Critical	AgentService, DB (mocked)
Formatters	test_formatters.py	5	High	None (pure)
Code Utils	test_code_utils.py	6	Medium	None (pure)
BACKEND INTEGRATION		12		
API + WebSocket	test_api.py	12	Critical	In-memory SQLite, TestClient
FRONTEND UNIT		93		
lib/utils	utils.test.ts	6	Low	None (pure)
lib/constants	constants.test.ts	5	Low	None (read-only)
lib/parseStatusInfo	parseStatusInfo.test.ts	14	High	None (pure)
lib/formatStatusContent	formatStatusContent.test.ts	8	High	None (pure)
lib/javaFormatter	javaFormatter.test.ts	10	Medium	None (pure)
lib/indentation	indentation.test.ts	8	Medium	None (pure)

lib/buildMetrics	buildMetrics.test.ts	6	Medium	None (pure)
lib/schemas (Zod)	websocket.test.ts	8	Medium	None (pure)
store/useChatStore	useChatStore.test.ts	14	Critical	fetch (mocked)
hooks/ useOrchestrationSocket	useOrchestrationSocket.test.tsx	10	Critical	WebSocket, store
ErrorBoundary	ErrorBoundary.test.tsx	4	Medium	React rendering
FRONTEND INTEGRATION		6		
Terminal	Terminal.test.tsx	3	High	Store + RTL
ChatWorkspace	ChatWorkspace.test.tsx	3	High	Store + Hook + RTL
		281 total		

Legend

Type	Meaning
Positive	Valid input, correct output
Negative	Invalid input, expected failure
Edge	Boundary conditions, empty/null, extreme values
Critical	Core pipeline logic — must pass for system correctness
High	Important module — should pass before merge
Medium	Utility module — good to have passing
Low	Trivial module — informational pass/fail

Document version 1.0 — Last updated June 24, 2026